The goal of the final project is to experiment with different algorithmic solutions in Reinforcement Learning, and with the training and optimization of the various hyperparameters of the algorithms.

The general framework is an escape-room game. You must create an escape room that contains at least 4 different rooms. In each room, a different algorithm and a different task must be run. In each room, there is a single final state that allows the agent to exit the room and move on to the next room. The faster the agent succeeds in solving the escape room, the higher its reward will be.

The first three rooms will be based on a 10x10 grid.

Known environment model:

1. **The first room** is a room in which the model of the environment is known. Therefore, we will apply the **Dynamic Programming** algorithm in this room. This room must include several slippery cells.

Unknown environment model:

2. **In the second room**, we will apply the **SARSA** algorithm. This room will also include several slippery cells.
3. **In the third room**, we will apply the **Q-Learning** algorithm.

Unknown model, function approximation:

4. **In the fourth room**, the game is not grid-based, and the state is defined by **X, Y, Vx, Vy**. The size of the room is **10x10 meters**. The movement is continuous: the player chooses the direction of movement at every time unit, which is **0.02 seconds**. The player's velocity is discrete: **Vx, Vy ∈ {-1, 0, 1}**.
5. **Optional:** The fifth room is a room in which there are obstacles that the player must avoid on the way to the exit. The width of the obstacles is **0.5 meters**. The number and location of the obstacles will be dynamic. You must provide control over the agent's **observation**: in addition to its own dynamics — position and velocities — the agent will be able to see obstacles **X meters ahead**. The distance is measured from the center of the character to the center of the obstacle. At the end of the training, it is possible to generate a random room and test the learned policy in it.

In each of the rooms, it should be possible to control all the parameters of the algorithm and the training stages. The application must save and display graphs that track the progress of learning and exploration throughout the training process.

At the end of the training, there should be an option to replay individual episodes in order to understand what the algorithm learned at different stages.

The tasks in the rooms should be built with an increasing level of difficulty, either in terms of task complexity or the number of actions required. Dynamic components may also be added, such as enemies, shortcuts, rewards, and traps.

The system code must be written in **Python**. It is recommended that the interface be built in **HTML**.

Project grading:

A basic project that meets the basic requirements of the final project will receive a grade of **80**.

In order to receive a higher grade, the project must demonstrate the ability to handle challenging and creative tasks, the use of a variety of algorithms, and a variety of solutions.

A **README** file must be attached, explaining for each room the structure of the states and rewards, as well as the parameters that were suitable for solving the problem in the most optimal way.